Technical Project Report: Smart IoT RFID Attendance System

Course/Project Code: final-year-project-v1

Project: Smart IoT RFID Attendance System — Real-Time Cloud-Connected Dashboard

Status: Completed & Deployed

Date: June 6, 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

FINAL YEAR CAPSTONE PROJECT REPORT

Table of Contents

Abstract

Traditional manual attendance marking is time-consuming, prone to human error, and vulnerable to proxy attendance. This report details the design and implementation of the **Smart IoT RFID Attendance System**, an integrated solution leveraging IoT hardware and cloud technologies. The system utilizes an ESP32 microcontroller paired with an MFRC522 RFID reader to capture student scans, instantly transmitting raw data to a Firebase Firestore database via a Cloud Run API endpoint. A premium, real-time web dashboard built using React 19, Tailwind CSS v4, and Recharts monitors student check-ins, visualizes attendance trends, manages student profiles, and generates daily, weekly, or monthly PDF and Excel reports. The result is a robust, paperless, and low-latency school/corporate attendance environment.

Chapter 1: Introduction & Project Overview

1.1 Problem Statement

In educational institutions and corporate workplaces, recording attendance is a fundamental daily process. Traditional methods involving paper registers or manual callouts suffer from: * **Inefficiency:** Substantial classroom time is wasted calling out roll numbers. * **Accuracy Issues:** High probability of clerical mistakes, illegible handwriting, or lost registers. * **Proxy Attendance:** Students sign in for absent classmates, undermining attendance policies. * **Lack of Real-Time Visibility:** Administrators, parents, and instructors have no instant way to view daily presence figures or generate automatic reports.

1.2 Project Objectives

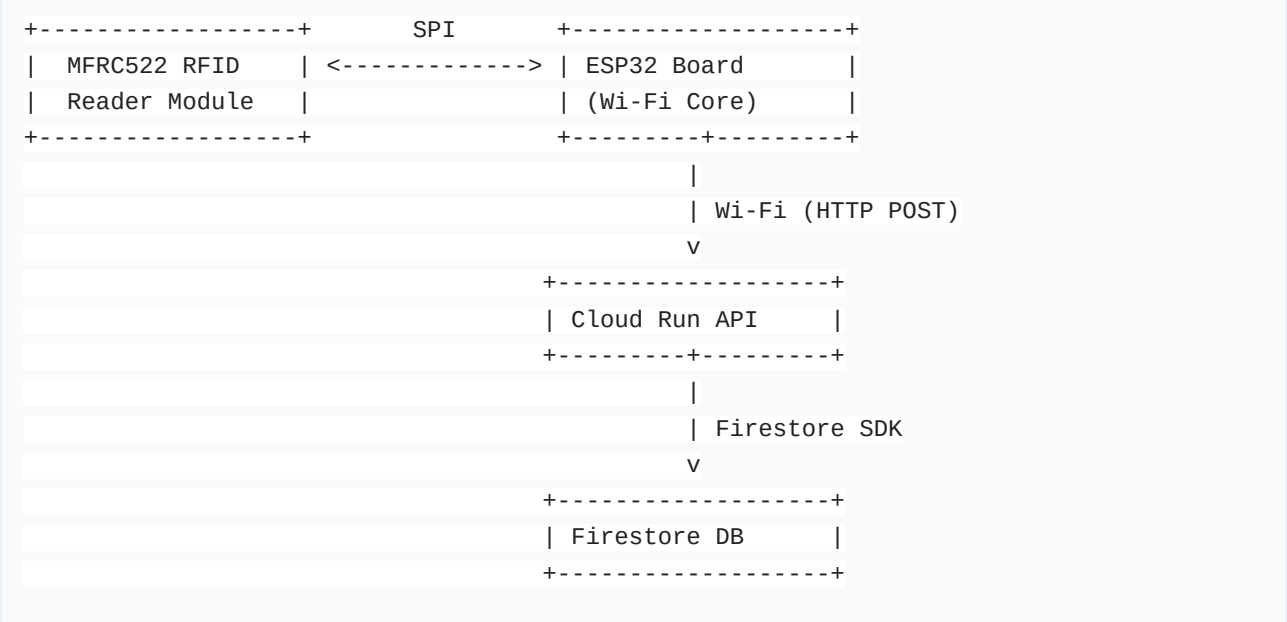
To resolve these issues, this project implements a hardware-software integrated IoT system aiming to: 1. **Automate Logging:** Register check-ins in under a second using unique radio-frequency identifiers (RFID). 2. **Synchronize Instantly:** Store scan records dynamically in a highly available cloud database. 3. **Enhance User Experience:** Provide a sleek, glassmorphic administrative interface showing live check-in feeds. 4. **Facilitate Analysis:** Generate analytics charts detailing peak scanning times, low attendance subjects, and exportable reports for administrative audits.

1.3 System Scope

The system handles: * RFID scanning on campus gates or classroom doors using low-cost ESP32 IoT boards. * Automated student lookups by mapping raw RFID UIDs to student records in the database. * Admin controls including student profile updates, CSV roster uploads, real-time logs, chart visualizations, report generation, and security access controls.

Chapter 2: Hardware Architecture & Working Principles

The physical layer consists of an edge node that reads RFID cards and sends data payloads over Wi-Fi.



2.1 ESP32 Microcontroller Overview

The **ESP32** is a low-cost, low-power system on a chip (SoC) with integrated Wi-Fi and dual-mode Bluetooth. It operates at 240 MHz with a dual-core Xtensa 32-bit microprocessor. In this architecture, it handles: * Establishing and maintaining Wi-Fi connections. * Initializing the SPI bus to communicate with the RFID reader. * Encapsulating the RFID card UID in an HTTP POST request payload. * Receiving host responses and triggering indicator LEDs (Green for valid scan, Red for network/API error).

2.2 MFRC522 RFID Reader Module

The **MFRC522** is a highly integrated reader/writer for contactless communication at 13.56 MHz. It utilizes an outstanding modulation and demodulation concept for passive contactless communication methods. * **Operating Frequency:** 13.56 MHz. * **Communication Protocol:** Serial Peripheral Interface (SPI). * **RFID Tags:** ISO/IEC 14443A cards and key fobs.

2.3 Wiring Diagram & SPI Pin Connections

The connections between the MFRC522 module and the ESP32 development board are detailed below:

MFRC522 Pin	ESP32 GPIO Pin	Description

VCC	3.3V	Power supply (MFRC522 requires 3.3V, not 5V)
RST	GPIO 22	Reset pin
GND	GND	Ground connection
MISO	GPIO 19	Master In Slave Out (SPI data)
MOSI	GPIO 23	Master Out Slave In (SPI data)
SCK	GPIO 18	Serial Clock (SPI clock)
SDA (SS)	GPIO 5	Slave Select / Chip Select

2.4 Hardware Firmware & Data Transmission Flow

The ESP32 firmware executes the following sequential logic: 1. **Setup Phase:** - Initialize Serial monitor at 115200 baud. - Configure SPI bus parameters. - Initialize MFRC522 module utilizing the `MFRC522` Arduino library. - Establish Wi-Fi connection with configured credentials. 2. **Main Loop Phase:** - Check if a new RFID card is present in the reader's field. - Read the serial number (UID) of the card. - Format the UID to a hexadecimal string representation. - Construct a JSON payload: `{"uid": "A3 B2 C5 D9", "subject": "Distributed Systems"}`. - Check Wi-Fi status. If disconnected, try reconnecting automatically. - If connected, initiate an HTTP client request to the Cloud Run API endpoint: - URL: `https://api-rfid-attendance.run.app/scan` - Header: `Content-Type: application/json` - Method: `POST` - Payload: JSON string - Await response: - If HTTP Status code is `200` (OK) or `201` (Created), flash a Green LED for 200ms and beep the passive buzzer once. - If error (e.g. `404` Student Not Found, or `500` Server Error), flash a Red LED three times. - Halt card reading to prevent duplicate scans of the same card in immediate succession (e.g. 2-second delay).

Chapter 3: Software Implementation & Frontend Architecture

The administrative user interface is designed as a single-page application focused on high visual appeal and immediate updates.

3.1 Frontend Tech Stack

- **Vite:** Build tool ensuring ultra-fast compilation and Hot Module Replacement (HMR).
- **React 19:** View engine leveraging functional components, state hooks, and side-effects.
- **Tailwind CSS v4:** Modern styling framework using native utility classes, absolute CSS variables, and modern glassmorphism.
- **Recharts:** Declarative chart graphics rendered dynamically over SVGs.
- **Framer Motion:** Custom keyframe animators managing entry and exit transitions of cards and modals.
- **jsPDF + AutoTable:** Local JavaScript libraries generating reports directly within the client browser.
- **xlsx (SheetJS):** Formats student data structures into standard binary Excel spreadsheet sheets.

3.2 Project Directory Structure

The frontend application follows a structured layout separation: * README.md: Primary system instructions. * `src/` * main.jsx: Application bootstrapper. * App.jsx: Defines routes and main layout frame. * index.css: Design utility overrides and glassmorphism. * `firebase/` * config.js: Initializer settings for the Firestore SDK. * `hooks/` * useAttendance.js: Real-time collection synchronization. * useStudents.js: Controls student profiles with Firestore bindings. * useStats.js: Reactive aggregator computing metrics for layout panels. * `components/` * `reports/` * ReportGenerator.jsx: Filter and export actions (PDF/Excel). * `ui/`: Reusable UI modules (Modals, Badges, Search Inputs). * `pages/` * DashboardPage.jsx: Admin statistics dashboard. * StudentsPage.jsx: Student rosters. * AttendancePage.jsx: Live logs database. * AnalyticsPage.jsx: Recharts trend components. * ReportsPage.jsx: Wraps report parameters. * SettingsPage.jsx: Configurations and server nodes check.

3.3 Styling System: Tailwind CSS v4 & Glassmorphism

The user experience stands out due to its modern **SaaS-style dark dashboard design** implemented via `index.css`. Main aesthetics include: * **Dark Background:** Consistent `#09090b` (Zinc-950) to minimize eye strain. * **Glassmorphic Cards:** Glass elements using white borders with low opacity (`border-white/5`),

translucent backgrounds (`bg-dark-800/30`), backdrop blurs, and subtle drop shadows. * **Harmony Colors:** Accentuation with HSL tailwinds: Primary Emerald (`text-emerald-400`), alerts (`text-red-400`), and highlights (`text-primary-400`).

3.4 Custom React hooks for Real-Time State Management

By implementing custom hooks, database integration is abstracted out of pages: * useStudents.js: Encapsulates query bindings, exposing helper routines like `addStudent`, `updateStudent`, and `removeStudent`. * useAttendance.js: Fetches the last 500 records on demand, registering real-time snapshots using `onSnapshot` to automatically refresh components when scans are recorded. * useStats.js: Uses `useMemo` to convert flat lists of students and scan records into computed parameters (e.g. today's presence, weekly progress chart datasets, subject attendance, and daily attendance percentages).

Chapter 4: Firebase Integration & Database Design

Firebase provides a reliable real-time serverless environment, minimizing backend maintenance overhead.

4.1 Firebase SDK Configuration

The client initializes firebase in config.js:

```
import { initializeApp } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';
import { getAuth } from 'firebase/auth';

const firebaseConfig = {
 apiKey: "AIzaSyCSt_blqmNmFo7KjodhLiWEZcweobgShys",
 authDomain: "rfid-attendance-system-498216.firebaseapp.com",
 projectId: "rfid-attendance-system-498216",
 storageBucket: "rfid-attendance-system-498216.firebasestorage.app",
 messagingSenderId: "334528977659",
 appId: "1:334528977659:web:aca25845df45fba06cee5e",
 measurementId: "G-185804REZT"
};

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const auth = getAuth(app);
export default app;
```

4.2 Firestore Collection Schema

Database storage is split into two primary document collections:

1. `students` Collection

Stores student profile metadata. The document ID is set to the student's unique RFID UID.

```
{
 "uid": "A3B2C5D9",
 "name": "Ankit Kumar",
 "rollNo": "CS2026042",
 "subject": "Distributed Systems",
 "status": "Active"
}
```

2. `attendance` Collection

Stores chronologically ordered scan records. Document IDs are randomly generated hashes.


```
{
 "id": "att_91823abce",
 "uid": "A3B2C5D9",
 "name": "Ankit Kumar",
 "rollNo": "CS2026042",
 "subject": "Distributed Systems",
 "status": "Present",
 "timestamp": "Timestamp(seconds=1780771200, nanoseconds=0)"
}
```

Chapter 5: Features & Functional Walkthrough

5.1 Admin Dashboard & Live Feed

When administrators log into the system, DashboardPage.jsx loads. * **Stat Cards:** Four distinct panels highlighting: 1. Total registered students. 2. Students marked present today. 3. Absent students count today. 4. Real-time Attendance Rate percentage. * **Live Scans Feed:** A sidebar that updates instantaneously when cards are scanned. Each card entry transitions in using Framer Motion animations and displays the student's name, timestamp, and card UID. * **System Health Monitor:** Displays green indicators showing internet and Firestore connection states.

5.2 Student Directory Management

The student module in StudentsPage.jsx supports full CRUD workflows: * **Add Student:** Form to register a student, binding a unique card UID, name, roll number, and subject. * **Edit/Delete:** In-row actions to modify parameters or delete student records. * **CSV Roster Upload:** Includes a parser mapping CSV headings to Firestore collections, facilitating batch registration of entire classroom cohorts. * **CSV Roster Export:** Exports student registry details directly.

5.3 Attendance Log with Search & Filter

The log in AttendancePage.jsx is optimized for quick audits: * **Search:** Matches input query string against student name, roll number, or card UID. * **Sort:** Sorts logs ascending or descending by timestamp. * **Pagination:** Restricts visibility to 15 entries per page, improving client render speeds.

5.4 Analytics & Trend Visualization

The analytics page AnalyticsPage.jsx converts tables of information into clean visual graphs: * **30-Day Trend Chart:** A line chart displaying daily presence rates to review weekend vs. weekday spikes. * **Subject-wise Distribution:** A bar chart showcasing attendance rates grouped by course subjects. * **Activity Leaderboard:** Highlights top-performing courses or students with consistent card scan rates.

5.5 Report Generator (PDF & Excel Exports)

The module ReportGenerator.jsx exports attendance data: * **Date-Range Filter:** Filters scan records for daily, weekly, or monthly intervals. * **PDF Export:** Uses `jsPDF-AutoTable` to compile filtered lists into a clean PDF document including header details, report duration metadata, presence stats, and an organized grid. * **Excel Export:** Converts datasets into binary sheets via `xlsx` formats, making it easy to share via email or upload to administrative portals. * **Direct Print:** Employs standard `@media print` CSS

configurations to isolate the report table and hide dashboard navigational borders for a clean print output.

5.6 System Settings & Security

Designed inside SettingsPage.jsx: * **Admin Profile settings:** Control authentication rules. * **Firebase Sync Panel:** Toggle state connections. * **ESP32 Node Status:** Monitors ping delays and activity timestamps of registered scanner hardware.

Chapter 6: Setup, Installation, and Deployment Instructions

Follow these instructions to run the project locally and host it online.

6.1 Local Development Setup

1. Install Node.js Dependencies

Navigate to the project directory and run the installer:

```
npm install
```

2. Configure Local Settings

Verify/replace Firebase variables inside config.js.

3. Run Development Server

Spin up the local dev server utilizing Vite:

```
npm run dev
```

Open http://localhost:5173 in your web browser.

6.2 Firebase Hosting Deployment

1. Download Firebase Command Line Interfaces (CLI)

Install global CLI tools:

```
npm install -g firebase-tools
```

2. Login & Initialize Project

```
firebase login
firebase init hosting
```

Select the corresponding Firebase console project id, designate `dist` as the public folder build, and configure the project structure as a Single-Page App (redirecting all paths to `index.html`).

3. Build & Deploy

Compile assets for production and push build files to Google's content delivery networks:

```
npm run build
firebase deploy --only hosting
```

Chapter 7: Conclusion & Future Scope

7.1 Achievements

The project successfully bridges physical IoT nodes (ESP32/RFID) with cloud data platforms and real-time frontend dashboards. The glassmorphic web portal loads in milliseconds, synchronizes check-ins in under one second, and reduces administrative overhead by offering automated, clean reports in PDF and Excel formats.

7.2 Limitations

- **Physical Proximity Required:** Students must hold their passive cards directly next to the scanner coil.
- **Power Source Dependency:** If power is cut to the ESP32 scanner node, attendance recording goes offline.
- **Card Theft/Misplacement:** Does not prevent one student from carrying and scanning multiple cards.

7.3 Future Enhancements

- **Biometric Multi-Factor Authentication:** Combine RFID scans with fingerprint modules or cameras executing local facial recognition on the ESP32 node.
- **Power Over Ethernet (PoE):** Replace standard Wi-Fi ESP32 modules with Ethernet-enabled alternatives, ensuring stable power delivery and higher network availability.
- **Push Notifications:** Configure Cloud Functions to notify parents or administrators via SMS or Telegram immediately when a student scans their card.
- **Automatic Email Reports:** Run cron-job triggers scheduling Firestore Cloud Functions to email weekly attendance summaries directly to subject instructors.